Sistemas Embebidos

Práctica 2: Comunicaciones y Procesado

Objetivos

- Utilizar la conectividad Wifi del ESP32.
- Implementar un cliente MQTT para el envío de telemetría.
- Procesado en el Edge: Aplicar algoritmos de filtrado de datos (DSP básico) antes de enviar los datos a la nube/servidor.

Descripción

En múltiples escenarios de IoT y sensorizacion en el edge aparece el problema de que los valores obtenidos pueden tener ruido. En nuestro caso, las señales biométricas del pulso del **MAX30105** suelen ser ruidosas debido al movimiento del usuario o interferencias de luz ambiental. Enviar datos ruidosos a la nube, consume batería y satura el ancho de banda. Por eso limpiar la señal de pulso en el ESP32, extraer las pulsaciones reales y enviar solo datos procesados y fiables al servidor, es la práctica extendida cuando desarrollamos dispositivos de esta índole.

También veremos como utilizar el protocolo MQTT, un estándar en los dispositivos IoT de bajo consumo y como conectarnos a diferentes redes inalámbricas.

Parte A: Procesado de Datos (Edge Computing)

Utilizando la lectura del sensor MAX30105 de la Práctica 1, vamos a desarrollar nuestro propio algoritmo de filtrado de señales y a compararlos con los datos en crudo. Implementa los siguientes filtros:

- Filtro de Media Móvil (SMA)
- Filtro Exponencial (EMA)
- Filtro de Mediana

Verifica que tus lecturas son estables moviendo el dedo por el sensor.

Es habitual en el IoT que un dispositivo esté alimentado por batería, esta limitación nos impide utilizar la comunicación (uno de los procesos que más energía consume) sin control. Es habitual tener que programar estrategias para evitar saturar las redes y vaciar la batería demasiado rápido, para ello vamos a usar estas estrategias:

- Downsampling (Submuestreo): El sensor puede leer muchos valores por minuto, pero nosotros solo usaremos un valor promedio cada 30 segundos. Implementa este método en tu código y almacena el valor para la siguiente parte.
- Umbral de cambio: Solo enviar un mensaje MQTT si el valor cambia significativamente (por ejemplo, si el pulso sube o baja más de 5 BPM). Si el paciente está estable, solo se envía la muestra habitual. Implementa en tu código un valor de threshold (umbral) con valor 5 y utilízalo en el bucle de lecturas para forzar un envío en la siguiente apartado.

- Análisis de Alarmas: Si se llega a ciertos niveles, Ej: BPM > 160. Haz que el dispositivo se prepare para enviar una alarma. Para evitar que la alarma se encienda y apague constantemente si el valor oscila justo en el límite, implementa un timeout entre alarmas de un minuto.

Implementa en tu código estos cambios y muestra ahora por pantalla solo el valor que se fuese a enviar. Muestra si el valor viene por el apartado A, B o C mostrando también este carácter.

Parte B: Comunicación IoT (MQTT)

Ahora que ya tenemos listo el dato que queremos enviar, tenemos que preparar nuestro script para que se conecte a una red wifi, (El esp32 no podrá conectarse a eduroam). Necesitarás crear un hotspot con tu móvil o conectarte a la red facilitada por el profesor. Cuando tengamos acceso a internet, vamos a conectarnos a nuestro servidor MQTT para enviar los datos filtrados.

Paso 1: Gestión Segura de Credenciales (WiFi)

Para evitar subir nuestras contraseñas a repositorios públicos, utilizaremos un archivo de cabecera independiente.

1. En tu proyecto, crea un nuevo archivo llamado arduino_secrets.h.
2. En tu script principal, cárgalo usando #include "arduino_secrets.h" y utiliza las constantes SECRET_SSID y SECRET_PASS en la función WiFi.begin().

Paso 2: Diagnóstico de Red del Dispositivo

Antes de enviar datos, debemos verificar que el dispositivo está correctamente identificado en la red, . Carga o prepara un script que inicialice la conexión WiFi y, una vez conectado, imprima por el Monitor Serial la siguiente información:

- **Dirección IP** asignada.
- **Puerta de enlace** (Gateway).
- **Dirección MAC:** (Identificador único físico de tu ESP32).
- **Fuerza de la señal (RSSI):** Para verificar la calidad del Hotspot.

Extra: Haz que esta información también se muestre brevemente, 3 segundos, por la **pantalla OLED** antes de pasar al modo de monitorización de tu script final.

Paso 3: Conexión a un Broker MQTT Público (Prueba inicial)

El protocolo MQTT requiere un "servidor" intermedio llamado **Broker**. Primero probaremos la conectividad con uno abierto.

1. Instala la librería PubSubClient.
2. Configura el cliente para conectarse a broker.hivemq.com o test.mosquitto.org en el puerto 1883. Puedes tener errores de conexión si estás en eduroam.
3. **Suscripción:** Suscríbete a un topic de prueba (ej. clase/test) y verifica en el Monitor Serial que recibes mensajes enviados desde un cliente externo como MQTT Explorer.

Paso 4: Envío de Telemetría al Broker de Clase

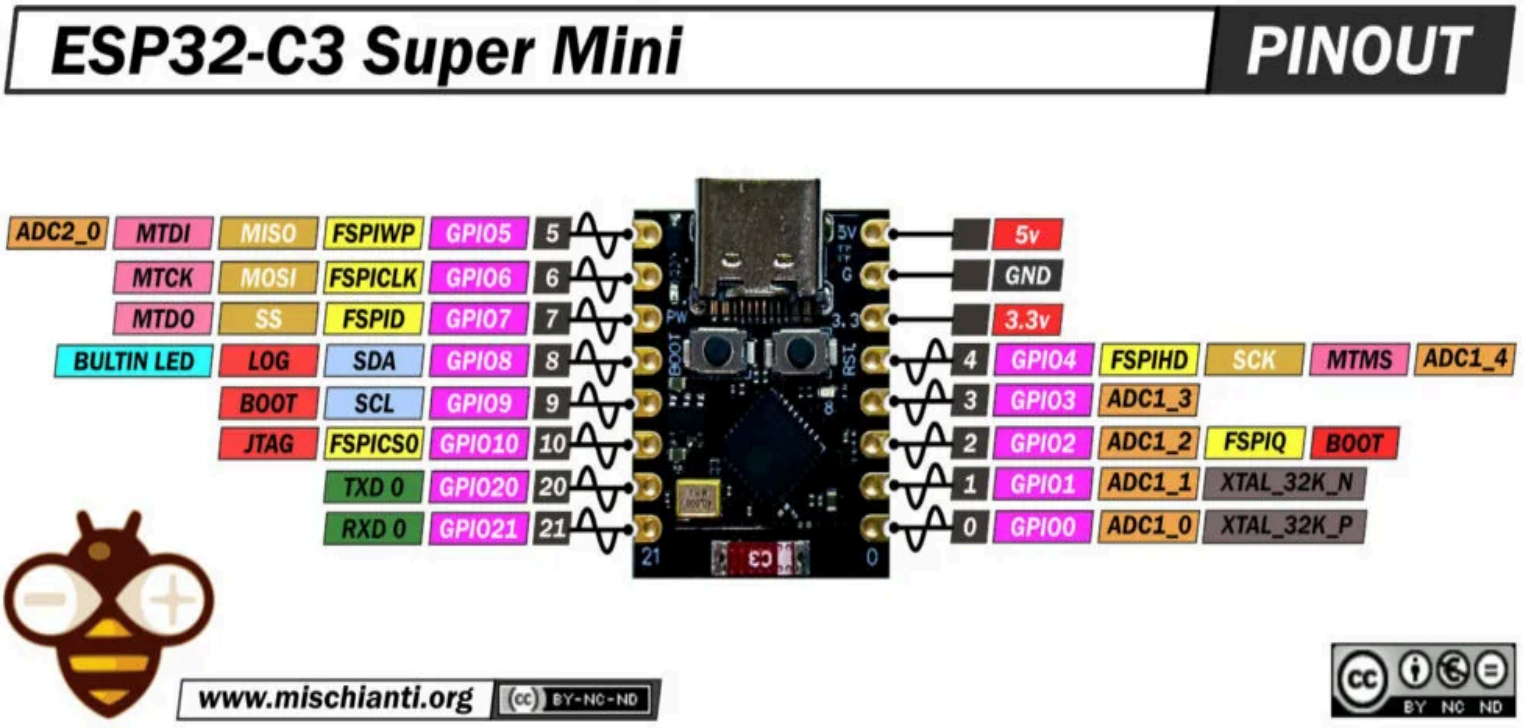
Una vez validada la conexión, es hora de enviar los datos reales de nuestra práctica

1. **Configuración del servidor:** Cambia los parámetros de conexión por los facilitados por el profesor en la sesión de prácticas.
2. **Publicación:** Envía el valor de las pulsaciones obtenido al topic: clase/salud/[vuestras_inciales]/datos. Comprueba desde el panel si el dato está llegando. Envía solo el valor numérico de las pulsaciones, por ejemplo, 55.
3. **Suscríbete:** Prueba suscribirte a un topic, por ejemplo clase/salud/[vuestras_inciales]/ordenes desde tu microcontrolador y comprueba que recibes el mensaje que mandas desde el panel.

Alternativa: Servidor Local con Docker

Si prefieres no usar el broker de la clase o el público está saturado, puedes levantar tu propio broker en tu PC si tienes docker instalado en tu PC. Descarga de moodle la carpeta del proyecto y ejecuta la siguiente orden:

- Ejecuta en tu terminal: **docker-compose up -d => http://127.0.0.1:1880/ y http://127.0.0.1:1880/dashboard/page1**
- Ahora, en el script del ESP32, la dirección del broker será la **IP local de tu ordenador**.

Aclaraciones para la práctica:

Si en algún momento no llegas a leer los mensajes del serial que ocurren durante el setup() añade el siguiente código después de Serial.begin(9600); while(!Serial); de esta forma el programa esperará a que Serial esté disponible para ser usado antes de continuar.